Übungsblatt 9: Integrierte Fallstudie und Timing-Verifikation

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: Ein vollständiges Zeitmodell (8 Punkte)

Eine lokale Überwachung besitzt drei Ausführungspfade auf **einem** Prozessor. Es gelten feste Prioritäten und Präemption. Alle Pfade werden erstmals bei null freigegeben, danach periodisch. Die relative Deadline ist jeweils die Periode. Ressourcenblockierung, Interrupt-, Wechsel- und Verwaltungskosten sind im folgenden Lehrmodell null. C bezeichnet eigene Prozessorzeit, Q_S das Serverbudget und T die Periode; alle Zeiten sind in ms angegeben.

Pfad	Prozessorzeit oder Budget	Periode	Priorität
Regler R	$C_R = 1$	$T_R = 6$	höchste
Polling Server S	$Q_S = 3$	$T_S = 12$	mittlere
Monitor M	$C_M = 2$	$T_M = 24$	niedrigste

Der Server bedient Anfragen in FIFO-Reihenfolge, also in der Reihenfolge ihrer Annahme. Jede Anfrage benötigt genau 1 ms Prozessorzeit. Für die Antwortzeitanalyse wird der Server konservativ als periodische Last mit $C_S = Q_S$ betrachtet. Die Antwortzeititeration lautet hier

$$R_i^{(k+1)} = C_i + \sum_{h \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_h} \right\rceil C_h, \qquad R_i^{(0)} = C_i.$$

$hp(i)$ enthält die höher priorisierten Pfade. Die Aufrundung zählt deren mögliche Freigaben im betrachteten Antwortintervall.

1. Berechnen Sie die gesamte reservierte Auslastung U. (2 Punkte)
2. Bestimmen Sie die Antwortzeit des Reglers sowie die Fixpunkte für Server und Monitor. Zeigen Sie die Zwischenwerte. (4 Punkte)
3. Prüfen Sie die drei relativen Deadlines. Warum ist der Serverfixpunkt noch keine Antwortzeitgrenze für jede beliebige Anfrage? (2 Punkte)

Musterlösung: Die reservierte Auslastung beträgt

$$U = \frac{1}{6} + \frac{3}{12} + \frac{2}{24} = \frac{2}{12} + \frac{3}{12} + \frac{1}{12} = \frac{1}{2} = 50\,\%.$$

Der Regler hat keine höhere Störung: $R_R = C_R = 1$ ms. Für den Server gilt

$$R_S^{(0)} = 3, \qquad R_S^{(1)} = 3 + \left\lceil \frac{3}{6} \right\rceil = 4, \qquad R_S^{(2)} = 3 + \left\lceil \frac{4}{6} \right\rceil = 4.$$

Für den Monitor werden beide höheren Pfade berücksichtigt:

$$\begin{aligned} R_M^{(0)} &= 2, \\ R_M^{(1)} &= 2 + \left\lceil \frac{2}{6} \right\rceil 1 + \left\lceil \frac{2}{12} \right\rceil 3 = 6, \\ R_M^{(2)} &= 2 + \left\lceil \frac{6}{6} \right\rceil 1 + \left\lceil \frac{6}{12} \right\rceil 3 = 6. \end{aligned}$$

Damit sind die Fixpunkte $R_R = 1$**,** $R_S = 4$**,** $R_M = 6$ **ms**. Am rechten Rand wird ein bei genau 6 ms freigegebener höherer Job nicht zusätzlich in das halboffene Antwortintervall $[0, 6)$ gezählt: Der Monitor kann bei 6 ms gerade fertig sein.

Es gelten $1 \leq 6$, $4 \leq 12$ und $6 \leq 24$; die periodischen Modellfristen sind erfüllt. Zwei Punkte für die Lastrechnung, ein Punkt für den Regler und je eineinhalb Punkte für die beiden nachvollziehbaren Iterationen. Je ein weiterer Punkt für die Deadlinebewertung und die folgende Unterscheidung: Eine aperiodische Anfrage wartet möglicherweise auf den nächsten Poll und auf ältere Anfragen. R_S beschreibt die Antwort eines aktivierten Serverjobs mit Budget Q_S, nicht diese zusätzliche Wartezeit. $U < 1$ allein ersetzt die Antwortzeitanalyse ebenfalls nicht.

Aufgabe 2: Polling mit einer endlichen Anfragefolge (8 Punkte)

Es gilt das in dieser Aufgabe erneut vollständig angegebene Modell: Regler $C_R = 1, T_R = 6$ ms mit höchster Priorität, Polling Server $Q_S = 3, T_S = 12$ ms mit mittlerer Priorität, Monitor $C_M = 2, T_M = 24$ ms mit niedrigster Priorität. Freigaben beginnen bei null. Ein Kern, Präemption, keine Blockierung und keine Zusatzkosten. Jede Anfrage kostet genau 1 ms Prozessorzeit.

Der Server prüft die FIFO-Queue **logisch bei** $0, 12, 24, \ldots$ **ms**. Ist sie dann leer, verfällt das gesamte Budget dieser Periode. Nach Abschluss der aktuellen Anfrage endet die Bedienung, wenn keine weitere Anfrage wartet; dabei verfällt Restbudget. Das Entnehmen der letzten Nachricht bricht ihre laufende Verarbeitung nicht ab. Während aktiver Bedienung dürfen neue Ankünfte noch vorhandenes Budget nutzen. Neue Anfragen aktivieren keinen bereits beendeten Serverjob erneut. Bei einem zeitgleichen Poll wird eine Ankunft zuerst eingeordnet. Die Queue hat vier Plätze und ist anfangs leer. A, B, C und D treffen bei 2, 3, 4 und 5 ms ein; weitere Anfragen fehlen.

1. Zeichnen Sie die Prozessorbelegung im halboffenen Intervall $[0, 28)$ ms. Kennzeichnen Sie die Serverbedienungen und den leeren Poll bei null. (4 Punkte)
2. Berechnen Sie die vier Anfrageantworten jeweils von Ankunft bis Abschluss. (3 Punkte)
3. Warum darf D nicht in derselben Bedienrunde wie A, B und C fertig werden? (1 Punkt)

Musterlösung: Beim Poll bei null ist die Queue leer. Der Server verliert sein Budget; die späteren Ankünfte können es nicht zurückholen. Die vier Plätze reichen für A bis D.

Intervall in ms	Prozessorbelegung
$[0, 1)$	Regler
$[1, 3)$	Monitor

Intervall in ms	Prozessorbelegung
$[3, 6)$	Leerlauf
$[6, 7)$	Regler
$[7, 12)$	Leerlauf
$[12, 13)$	Regler
$[13, 14)$	Server: A
$[14, 15)$	Server: B
$[15, 16)$	Server: C
$[16, 18)$	Leerlauf
$[18, 19)$	Regler
$[19, 24)$	Leerlauf
$[24, 25)$	Regler
$[25, 26)$	Server: D
$[26, 28)$	Monitor

Der Monitor bei null läuft trotz der Ankünfte weiter: Ankunft ist hier keine erneute Serverfreigabe. Die Antworten sind

$$R_A = 14 - 2 = 12, \quad R_B = 15 - 3 = 12, \quad R_C = 16 - 4 = 12, \quad R_D = 26 - 5 = 21 \text{ ms}.$$

Nach drei Anfragen sind die drei Millisekunden Budget vollständig verbraucht. Dass danach Prozessorzeit frei wäre, berechtigt D nicht zum Weiterlaufen; erst die nächste Pollgelegenheit bei 24 ms eröffnet neues Budget. Vier Punkte für die vollständige Reihenfolge einschließlich Monitor und Regler, drei Punkte für die vier Antworten, ein Punkt für die Budgetbegründung.

Aufgabe 3: Puffer, Filter und Zustandsanzeige (8 Punkte)

1. Eine FIFO-Queue hat drei Plätze und ist anfangs leer. Führen Sie nacheinander aus: Put 1, Put 2, Get, Put 3, Put 4, Put 5, Get. Put bedeutet nichtblockierendes Einfügen, Get Entnehmen; gleiche Nachrichtenpriorität ist vorausgesetzt. Geben Sie die Queueinhalte nach jeder Operation, die entnommenen Werte, Annahmen und Abweisungen an. Prüfen Sie am Ende $I = A + D$ und $A = Q + P$. Dabei sind I die Einfügeversuche, A die Annahmen, D die Abweisungen, Q der Restpuffer und P bereits abgeschlossene Verarbeitungen. Jedes Get wird hier sofort vollständig verarbeitet. (4 Punkte)
2. Unabhängig davon wird die tatsächlich verarbeitete Folge 20,100,160,40,10,10 gefiltert. Berechnen Sie den gleitenden Mittelwert über höchstens die letzten drei vorhandenen Werte, anfangs ohne Nullauffüllung. Die nichtnegative Ganzzahldivision rundet ab. Wenden Sie anschließend eine Hysterese an: hoch ab 90, niedrig bei höchstens 60, im Zwischenband alten Zustand halten; Anfangszustand niedrig. Geben Sie alle Filterwerte und Zustände an. (4 Punkte)

Musterlösung: Die Queue wird wie folgt verfolgt:

Operation	Inhalt danach	Ergebnis
Put 1	1	angenommen
Put 2	1,2	angenommen
Get	2	1 verarbeitet
Put 3	2,3	angenommen
Put 4	2,3,4	angenommen

Operation	Inhalt danach	Ergebnis
Put 5	2,3,4	voll, 5 abgewiesen
Get	3,4	2 verarbeitet

Es gelten $I = 5, A = 4, D = 1, Q = 2, P = 2$: $5 = 4 + 1$ und $4 = 2 + 2$. Zwei Punkte für die Zustandsfolge und zwei für Ergebnis und Bilanzen. Ein zeitlich verteilter Burst mit anderen Gets hätte nicht automatisch dieselbe Dropzahl.

Eingang	verwendetes Fenster	Rechnung	Filter	hoch
20	20	20/1	20	0
100	20,100	120/2	60	0
160	20,100,160	280/3 abgerundet	93	1
40	100,160,40	300/3	100	1
10	160,40,10	210/3	70	1
10	40,10,10	60/3	20	0

Die Division 280/3 ergibt 93 mit Rest 1. 70 liegt zwischen den beiden Grenzen und erhält den vorherigen hohen Zustand. Zwei Punkte für nachvollziehbare Filterwerte, zwei für die richtige Hysterese einschließlich Anfang und Zwischenband. Das Fenster enthält nur tatsächlich verarbeitete Werte; unbekannte verworfene Eingaben werden nicht erfunden.

Aufgabe 4: Zeitbefund und Fehlerreaktion (8 Punkte)

Alle Zahlen sind Lehrannahmen. Die Zeitpaare benutzen dieselbe Modelluhr.

1. Drei vollständig erfasste Anfragen haben Ankunft/Abschluss in ms: $(0, 7)$, $(30, 39)$ und $(60, 70)$. Berechnen Sie Antworten, beobachtetes Maximum und Antwortzeitspanne. Bewerten Sie eine Grenze von 9 ms. Ein vierter Datensatz besitzt keinen Abschlusszeitpunkt; wie behandeln Sie ihn? (3 Punkte)
2. Ein Worker meldet den Abschluss vollständiger Kontrollrunden. Nach einem Hänger entstehen keine neuen Fortschrittsmarken. Eine weiter funktionierende Überwachung prüft periodisch mit $T_M = 24$ ms; ihre Antwortzeit bis zur abgeschlossenen Prüfung ist höchstens $R_M = 6$ ms, wie in Aufgabe 1 bestimmt. Die Frischeschwelle beträgt $\Theta = 22$ ms. Nach erkannter Veraltung benötigt die definierte Reaktion höchstens $T_A = 2$ ms. Geben Sie eine konservative Fehlerreaktionsgrenze ab Hänger an und prüfen Sie eine Anforderung von 55 ms. Der letzte gültige Fortschritt liegt spätestens beim Fehler; die Prüfung benutzt dessen tatsächliches Alter. Höher priorisierte Last bleibt innerhalb der analysierten Grenzen, sodass der Monitor seine Antwortzeit auch im Fehlerfall einhält. (3 Punkte)
3. Ein unabhängiger Watchdog wurde unmittelbar vor einem vollständigen Prozessorstillstand legal nachgeladen. Sein maximales Timeout beträgt 35 ms; danach dauert es höchstens 4 ms bis zum vereinbarten Reset-Ausgangszustand. Berechnen Sie die Grenze. Warum gilt diese Rechnung nicht unverändert für einen einzelnen hängenden Worker, während ein Supervisor noch läuft? (2 Punkte)

Musterlösung: Die Antworten sind $7 - 0 = 7$, $39 - 30 = 9$ und $70 - 60 = 10$ ms. Das beobachtete Maximum ist 10 ms, die Spanne $10 - 7 = 3$ ms. Die dritte Antwort verletzt die 9-ms-Grenze um 1 ms; die zweite erfüllt sie gerade. Der unvollständige vierte Datensatz liefert keine Antwortzeit und wird als fehlendes Zeitpaar dokumentiert, nicht als null. Je ein Punkt für Antworten, Maximum/Spanne und begründete Bewertung.

Die konservative Reaktion des weiter funktionierenden Supervisors ist

$$T_{\text{Fehler}} \leq \Theta + T_M + R_M + T_A = 22 + 24 + 6 + 2 = 54 \text{ ms}.$$

Die Schwelle kann gerade nach einer Prüfgelegenheit überschritten werden; daher kommen eine weitere Monitorperiode und dessen Antwort bis zur Entscheidung hinzu. Die Reaktion erfüllt die 55-ms-Modellanforderung mit 1 ms Abstand. Zwei Punkte für Ansatz und Rechnung, einer für die begründete Bewertung. Das ist kein gemessener Hardwarewert. Eine unendliche hochpriorisierte Busyloop, also eine ständig weiterlaufende Rechenschleife, würde diese Monitorannahme verletzen: Dann gilt die direkte Softwaregrenze nicht.

Beim vollständigen Prozessorstillstand unterbleiben alle weiteren Refreshs; $35 + 4 = 39$ ms sind die angenommene Grenze. Bei einem einzelnen Workerhänger kann der weiter laufende Supervisor zunächst noch nachladen, solange sein Frischekriterium den bisherigen Fortschritt akzeptiert. Dann muss auch die Zeit bis zum letzten zulässigen Refresh berücksichtigt werden. Ein Punkt für 39 ms, einer für die Unterscheidung. Ein blind aus einem Timerinterrupt nachgeladener Watchdog könnte einen Workerhänger sogar dauerhaft verdecken.

Aufgabe 5: Eine Abnahme mit gemischtem Ergebnis

Ein abgeschlossener Lauf hat 12 Einfügeversuche, 10 Annahmen, zwei gezählte volle-Queue-Abweisungen und zehn abgeschlossene Verarbeitungen; der Puffer ist leer. Eine gültig beobachtete Antwort beträgt 10 ms bei geforderten 9 ms. Der Kaltstart wurde noch nicht auf dem Board ausgeführt. Geben Sie getrennte Statuswerte für Datenbilanz, Antwortzeitforderung und Hardwarestart an. Weshalb ist „alles bestanden“ unzulässig?

Musterlösung: Datenbilanz konsistent: $12 = 10 + 2$ und $10 = 0 + 10$. Antwortzeitforderung verletzt um 1 ms. Boardkaltstart nicht geprüft. Ein logisch konsistenter Datenweg macht weder eine beobachtete Fristverletzung rückgängig noch einen nicht ausgeführten Hardwaretest erfolgreich. Die drei Statuswerte bleiben getrennt. Auch eine günstigere endliche Stichprobe wäre kein allgemeiner Nachweis der maximalen Ausführungszeit.

Aufgabe 6: Eine begrenzte Verarbeitung mit eigener Hysterese

Programmieren, etwa 30–40 Minuten auf einer vorbereiteten Basis. Verwenden Sie **NUCLEO-H755ZI-Q / STM32H755ZIT6**, STM32CubeMX und VS Code mit STM32 VS Code Extension. Es laufen genau zwei Anwendungsthreads auf CM4; CM7 bleibt bootdeaktiviert. Verwenden Sie die bereits geprüfte CM4-Startbasis aus Blatt 3 und die Vorbereitung zu Blatt 9. Board, B1, eingebaute LEDs und USB/ST-LINK genügen. Es gibt keine Dateien aus `code/lab09`, keinen gemeinsamen Speicherkanal und keinen Hardware-Watchdog in diesem Versuch.

Die Quelle erkennt einen stabilen B1-Tastendruck und bietet eine synthetische Nachricht mit nächster Eingabenummer an. Nummern werden **vor** dem nichtblockierenden Put vergeben. Die lokale Queue hat vier Plätze. Die Verarbeitung entnimmt auf einem 200-ms-Sollraster je Runde höchstens zwei Nachrichten. Das begrenzt die **Anzahl**; ohne nachgewiesene Einzelkosten und Budgetdurchsetzung ist dies kein CPU-Server mit garantierter Prozessorzeitreservation. Die Papiermodelle aus Aufgaben 1/2 und der reale Batchversuch werden getrennt bewertet. Im Programm wird die Queue bei den tatsächlichen Get-Aufrufen geprüft; eine während der Runde eintreffende Nachricht kann noch beim zweiten Get entnommen werden. Es gibt keinen zusätzlichen logischen Queue-Schnappschuss am nominellen Pollzeitpunkt.

Ihr Arbeitsauftrag

1. Schreiben Sie `Lab09Step`: Nullpointer, Nummer null, nicht steigende Nummern und Werte über 200 werden mit Rückgabe null abgewiesen, ohne den Zustand zu ändern. Bei gültiger Nachricht wird eine Nummernlücke gegenüber `last` verriegelt; anfangs gilt `last=0`. Hoch ab 90, niedrig bis 60, dazwischen alten Zustand halten. Erfolg liefert eins. Die Eingabenummern dieses Versuchs sind auf 1 bis 64 begrenzt und laufen nicht über.
2. Schreiben Sie die Schleife von `StartVerarbeitung`: pro Runde höchstens zwei erfolgreiche Gets mit Timeout null. Bei leerer Queue Runde beenden; andere Kernelrückgaben führen zu `Error_Handler`. Nach jeder gültigen Verarbeitung Zustandsanzeige und privates Protokoll aktualisieren. Warten erfolgt ausschließlich nach der Runde, über den vorgegebenen Helfer. Alle LED-Schreibzugriffe gehören diesem Thread.
3. Prüfen Sie zuerst die im Code angegebene private Folge mit Nummern 1,2,4,5 und Werten 25,110,75,45. Bestimmen Sie Sollzustand und Lückenanzeige. `Lab09CheckLocal` speichert die Ergebnisse in `lab09_test`; es simuliert keinen Boardlauf. Prüfen Sie zusätzlich eine wiederholte Nummer und einen ungültigen Wert auf unveränderten Zustand.
4. Bauen Sie die eigenen Funktionen, starten Sie auf dem Board und geben Sie einzelne B1-Tastendrücke mit genügend Abstand ein. **LD1/PB0** zeigt niedrig, **LD3/PB14** hoch, **LD2/PE1** eine verriegelte Eingabesequenzlücke. Quelle und Verarbeitung besitzen getrennte private Diagnosefelder. Lesen Sie diese erst nach kontrolliertem Anhalten; eine laufende Beobachtung mehrerer Felder ist kein atomarer Prüfpunkt.
5. Erklären Sie: Warum beweist eine gelbe LED weder die Zahl verworfener Eingaben noch deren Antwortzeit? Warum muss eine letzte Abweisung ohne spätere erfolgreiche Nachricht noch keine sichtbare Sequenzlücke erzeugen? Warum sind zwei Gets pro Runde kein Budget von zwei Millisekunden?

Vorbereitete Typen, Zustände und Helfer

Alle folgenden C-Bausteine werden im vorhandenen CM4-Modul der generierten Threadfunktionen in erhaltene Benutzerbereiche aufgenommen, beispielsweise `freertos.c` oder `app_freertos.c`. Keine zusätzliche Quelldatei wird benötigt. `main.h` enthält die CubeMX-Labels `LAB09_LOW` für PB0, `LAB09_GAP` für PE1, `LAB09_HIGH` für PB14 und `LAB09_BUTTON` für PC13. Die auf dem

tatsächlichen Board bestätigte aktive Tasterpolarität ist hier `GPIO_PIN_SET`. Die HAL ist die Hardware-Abstraktionsbibliothek von ST; CMSIS-RTOS2 ist die verwendete Kernelschnittstelle. Ein Kerneltick entspricht in dieser vorbereiteten Konfiguration 1 ms.

```
#include "main.h"
#include "cmsis_os2.h"
#include <stddef.h>
#include <stdint.h>

typedef struct { uint32_t seq, value; } Lab09Message;
typedef struct { uint32_t last, high, gap; } Lab09State;
typedef struct { uint32_t seq, value, high, gap; } Lab09Observation;

static osMessageQueueId_t lab09_queue;
/* Jeweils ein schreibender Thread; nur am Debughalt auswerten. */
static volatile uint32_t lab09_offered, lab09_accepted, lab09_dropped;
static volatile uint32_t lab09_processed, lab09_missed_round;
static volatile Lab09Observation lab09_log[64];
static volatile Lab09Observation lab09_test[4];

int Lab09Step(Lab09State *state, const Lab09Message *message);

static void Lab09WriteLEDs(const Lab09State *state)
{
    HAL_GPIO_WritePin(LAB09_LOW_GPIO_Port, LAB09_LOW_Pin,
                      state->high ? GPIO_PIN_RESET : GPIO_PIN_SET);
    HAL_GPIO_WritePin(LAB09_HIGH_GPIO_Port, LAB09_HIGH_Pin,
                      state->high ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LAB09_GAP_GPIO_Port, LAB09_GAP_Pin,
                      state->gap ? GPIO_PIN_SET : GPIO_PIN_RESET);
}

static void Lab09CheckLocal(void)
{
    const Lab09Message input[4] = {{1,25}, {2,110}, {4,75}, {5,45}};
    Lab09State state = {0};
    for (uint32_t i = 0; i < 4U; ++i) {
        if (!Lab09Step(&state, &input[i])) { Error_Handler(); }
        const Lab09Observation item = {
            input[i].seq, input[i].value, state.high, state.gap
        };
        lab09_test[i] = item;
    }
}

/* Ausschliesslich vom Verarbeitungsthread aufrufen. */
static void Lab09WaitRound(uint32_t *next)
{
    *next += 200U;
    const uint32_t now = osKernelGetTickCount();
    const uint32_t distance = *next - now;
    if (distance == 0U || distance > UINT32_C(0x7fffffff)) {
```

```
        ++lab09_missed_round;
        *next = now + 200U;
    }
    osStatus_t status = osDelayUntil(*next);
    if (status == osErrorParameter || status == osError) {
        /* Ziel bei Übergang in den Warteaufruf verpasst: einmal neu ausrichten. */
        ++lab09_missed_round;
        *next = osKernelGetTickCount() + 200U;
        status = osDelayUntil(*next);
    }
    if (status != osOK) { Error_Handler(); }
}
```

`Lab09State` gehört ausschließlich der Verarbeitung. `last` ist die letzte verarbeitete Eingabenummer, `high` und `gap` sind null oder eins. `lab09_test` und `lab09_log` werden nur von diesem Thread beschrieben. Die Quelldiagnosen `offered`, `accepted` und `dropped` besitzen dagegen ausschließlich den Quellthread als Schreiber; die Anwendung liest sie nicht threadübergreifend. `volatile` dient hier der Debuggerbeobachtung und ersetzt keine Synchronisation. Der Nachrichtenaustausch erfolgt durch die Queue.

`Lab09WaitRound` erhält das absolute Sollraster. Bei einem bereits erreichten/verpassten Ziel wird das nächste Ziel auf jetzt plus 200 Ticks gesetzt; es gibt keine sofortige Nachholserie. Läuft das Ziel erst im API-Übergang ab, wird einmal neu versucht; dabei werden sowohl `osErrorParameter` als auch `osError` berücksichtigt. Weitere Fehler führen zum Fehlerpfad. Die vorzeichenlose Differenz wird nur für Ziele mit weniger als einer halben 32-Bit-Zählerperiode Abstand ausgewertet. Das gilt bei 200 Ticks und den kurzen Versuchen. Der Zähler `missed_round` erfasst verpasste Warteziele und ersetzt keine separat definierte Jobdeadline. Eine verzögerte Wiederaufnahme bleibt möglich; `osDelayUntil` erzwingt keinen pünktlichen tatsächlichen Start.

Vorbereitete Quelle

Die Quelle pollt B1 nominal alle 10 Ticks. Drei aufeinanderfolgende gleiche Abtastungen übernehmen einen neuen stabilen Pegel. Nur dessen Übergang auf gedrückt erzeugt eine Nachricht. Das ist ein bewusst einfaches Entprellmodell; sehr kurze Tastendrücke können fehlen. Es wird weder eine EXTI-Unterbrechung noch eine physische Flankenzählung behauptet. Nach 64 angebotenen Nachrichten erzeugt die Quelle keine weiteren Nummern; ein Reset beginnt den Versuch neu.

```
void StartQuelle(void *argument)
{
    (void)argument;
    static const uint32_t values[4] = {25U, 110U, 75U, 45U};
    GPIO_PinState candidate = GPIO_PIN_RESET;
    GPIO_PinState stable = GPIO_PIN_RESET;
    uint32_t equal_samples = 0U;
    uint32_t seq = 0U;
    for (;;) {
        const GPIO_PinState raw = HAL_GPIO_ReadPin(
            LAB09_BUTTON_GPIO_Port, LAB09_BUTTON_Pin);
        if (raw != candidate) {
            candidate = raw;
            equal_samples = 1U;
```

```
        } else if (equal_samples < 3U) {
            ++equal_samples;
        }
        if (equal_samples == 3U && stable != candidate) {
            stable = candidate;
            if (stable == GPIO_PIN_SET && seq < 64U) {
                ++seq; /* Vor dem Put, auch wenn dieser abgewiesen wird. */
                const Lab09Message message = {seq, values[(seq - 1U) % 4U]};
                ++lab09_offered;
                const osStatus_t status = osMessageQueuePut(
                    lab09_queue, &message, 0U, 0U);
                if (status == osOK) { ++lab09_accepted; }
                else if (status == osErrorResource) { ++lab09_dropped; }
                else { Error_Handler(); }
            }
        }
        if (osDelay(10U) != osOK) { Error_Handler(); }
    }
}
```

Vorbereitete Kernelobjekte und Integrationspunkte

In CubeMX werden die beiden bisherigen Anwendungsthreads ersetzt: `Quelle` mit Eintritt `StartQuelle` und Priorität `osPriorityAboveNormal`, `Verarbeitung` mit Eintritt `StartVerarbeitung` und Priorität `osPriorityNormal`. Beide erhalten zunächst 1024 Byte CMSIS-Stackgröße; bei einem Words-Dialog mit 4-Byte-Elementen sind dies 256 Words. Heap und Stacks müssen im tatsächlichen Projekt geprüft werden. Kernelinterne Threads zählen nicht als zusätzliche Anwendungsthreads. Alte Default-/Laboranwendungen werden entfernt.

In der vorhandenen RTOS-Initialisierungsfunktion wird **nach `osKernelInitialize` und vor beiden Threadanlagen** genau diese Queue angelegt. Keine weitere gleichnamige generierte Queue konfigurieren:

```
if (osKernelGetTickFreq() != 1000U) { Error_Handler(); }
lab09_queue = osMessageQueueNew(4U, sizeof(Lab09Message), NULL);
if (lab09_queue == NULL) { Error_Handler(); }
```

Die beiden vorhandenen generierten `osThreadNew`-Aufrufe bleiben die einzige Anlage der Anwendungsthreads. Bei den genannten CubeMX-Tasknamen werden ihre tatsächlichen Handles nach Anlage und vor dem einzigen `osKernelStart` geprüft:

```
if (QuelleHandle == NULL || VerarbeitungHandle == NULL) {
    Error_Handler();
}
```

Die Namen in der tatsächlichen Generierung kontrollieren; keine zweiten Handles oder Threadanlagen daneben ergänzen. Erhalten Sie den bereits geprüften CM4-Systemstart und die gemeinsame HAL-/Kernel-SysTick-Zeitbasis aus Blatt 3. Der einzige Aufruf von `osKernelStart` und die Behandlung einer unerwarteten Rückkehr bleiben im generierten Startpfad. GPIO, Kernel und Queue müssen vor Nutzung bereit sein. Nur der Verarbeitungsthread schreibt LED-Pins; beim Systemstart initialisiert die GPIO-Konfiguration die Pins zunächst auf Low. `Error_Handler`

beendet bei Fehlern den Versuch durch einen dauerhaften Halt und kehrt nicht zurück; die tatsächlich generierte Funktion entsprechend prüfen.

Prüfprotokoll: Eigene Zustandsfunktion und Batchschleife, lokaler Funktionstest, Zielbuild, Kaltstart, Boardanzeige und Timing jeweils getrennt dokumentieren. Ein PDF- oder Hostlauf ist kein Boardtest. Ohne Messung bleiben reale Antwortzeiten und Prozessorzeitkosten offen. Eine schnelle Folge kann zwischen zwei sichtbaren Beobachtungen mehrere LED-Zustände durchlaufen; das private Protokoll ergänzt die Anzeige. Boardpinbelegung und aktive Polaritäten sind vorab gegen UM2408 zu prüfen.

Quellen: Buttazzo, *Hard Real-Time Computing Systems*; Arm CMSIS-RTOS2, Message Queue und Generic Wait Functions; ST UM2408. Alle Papierzeiten sind didaktische Annahmen. Die Programmieraufgabe demonstriert begrenzte Mengenverarbeitung und Zustandslogik, keine konkrete Watchdoghardware oder Sicherheitszertifizierung.

Musterlösung zu Aufgabe 6

Die Zustandsfunktion prüft zuerst alle Nachrichtenvoraussetzungen und verändert erst danach den Zustand. Ein mit null initialisierter, anschließend nur von dieser Funktion veränderter Zustand ist vorausgesetzt. Für den begrenzten Versuch ist `last+1` sicher: `last` bleibt zwischen null und 64. Die Hysterese enthält keine Warteoperation und schreibt selbst keine Hardware.

```
int Lab09Step(Lab09State *state, const Lab09Message *message)
{
    if (state == NULL || message == NULL || message->seq == 0U ||
        message->seq > 64U || message->seq <= state->last ||
        message->value > 200U) {
        return 0;
    }
    if (message->seq > state->last + 1U) { state->gap = 1U; }
    state->last = message->seq;
    if (message->value >= 90U) { state->high = 1U; }
    else if (message->value <= 60U) { state->high = 0U; }
    return 1;
}

void StartVerarbeitung(void *argument)
{
    (void)argument;
    Lab09CheckLocal();
    Lab09State state = {0};
    Lab09WriteLEDs(&state);
    uint32_t next = osKernelGetTickCount();
    for (;;) {
        for (uint32_t used = 0U; used < 2U; ++used) {
            Lab09Message message;
            const osStatus_t status = osMessageQueueGet(
                lab09_queue, &message, NULL, 0U);
            if (status == osErrorResource) { break; }
            if (status != osOK || !Lab09Step(&state, &message)) {
                Error_Handler();
            }
            Lab09WriteLEDs(&state);
            if (lab09_processed >= 64U) { Error_Handler(); }
            const Lab09Observation item = {
                message.seq, message.value, state.high, state.gap
            };
            lab09_log[lab09_processed] = item;
            ++lab09_processed;
        }
        Lab09WaitRound(&next);
    }
}
```

Die lokale Prüffolge liefert:

Nummer/Wert	hoch	Lücke	Begründung
1 / 25	0	0	untere Grenze, fortlaufende Nummer
2 / 110	1	0	obere Grenze erreicht
4 / 75	1	1	Zwischenband hält, Nummer 3 fehlt
5 / 45	0	1	untere Grenze, Lücke bleibt verriegelt

Ein konkreter Zusatztest nach dieser Folge benutzt zunächst `{5,110}` und dann `{6,201}`. Beide Aufrufe liefern null; `last=5, high=0, gap=1` bleibt unverändert. Ein Nullpointer wird ebenfalls ohne Änderung abgewiesen. Nach dem lokalen Test beginnt der reale Verarbeitungsthread mit einem eigenen frischen Zustand, während die Resultate in `lab09_test` erhalten bleiben.

Die Quellwerte beginnen wieder bei Nummer 1. Die gelbe LED zeigt eine beobachtete Eingabesequenzlücke; sie zählt nicht deren Größe. Im hier einzigen verlustbehafteten Queuepfad kann eine später empfangene höhere Nummer ausgelassene Angebote sichtbar machen. Die Queue-Abweisungszahl bleibt `lab09_dropped`. Wird die letzte angebotene Nachricht abgewiesen und kommt später keine weitere an, kann die Verarbeitung keine höhere Nummer und damit keine neue Lücke sehen. Aus einer nicht leuchtenden gelben LED folgt deshalb keine vollständige Verlustfreiheit.

Am kontrollierten Halt gelten nach abgeschlossenen Putversuchen $I = A + D$ mit `offered`, `accepted`, `dropped`. Für $A = Q + F + P$ muss zusätzlich der Bearbeitungszustand bekannt sein; mitten in der Entnahme/Protokollierung kann eine Nachricht in Bearbeitung sein. Die vorbereitete private Zustandsfolge ist kein realer Quelldatenlauf und wird nicht in diese Queuebilanz gezählt.

Zwei Entnahmen begrenzen die Menge pro Runde. Die Summe ihrer tatsächlichen Prozessorzeiten, die HAL-Ausgaben und die Verwaltung sind damit ohne weitere Schranken noch nicht in Millisekunden begrenzt. Ein echter Prozessorzeitserver erfordert einen passenden Kosten- und Durchsetzungsvertrag. Der Versuch enthält keinen Monitor und keinen Hardware-Watchdog; deren Fehlerzeitrechnungen aus Aufgabe 4 sind eigenständige Modellprüfungen.